```sql
-- 40. Insert a new college with name 'UIB', in the state 'IB', and with size 11500.
INSERT INTO COLLEGES(NAME, STATE, ENROLLMENT) VALUES ('UIB','IB',11500);

-- 41. Insert into APPLIES with college 'UIB', major 'IB', AND DECISION NULL all students
-- who didn't apply anywhere.
INSERT INTO APPLIES (SID, COLLEGE, MAJOR, DECISION)
SELECT s.id, 'UIB', 'IB', NULL
FROM STUDENTS s
WHERE NOT EXISTS (
   SELECT 1
   FROM APPLIES a
   WHERE SID = ID
);
-- 42. Admit to UIB EE all students who were refused (decision=false) in EE elsewhere.
INSERT INTO APPLIES (SID, COLLEGE, MAJOR, DECISION)
SELECT DISTINCT SID, 'UIB', 'EE', 1
FROM APPLIES
WHERE MAJOR = 'EE' AND DECISION = 0 AND COLLEGE != 'UIB';

-- 43. Delete all students who applied to more than two different majors.
-- 1. First delete all foreign keys from APPLIES table.
ALTER TABLE APPLIES
DROP CONSTRAINT FK_COLLEGE;
ALTER TABLE APPLIES
DROP  CONSTRAINT FK_SID;
-- 2. Add the foreign keys with an ON DELETE CASCADE CLAUSE
ALTER TABLE APPLIES
ADD CONSTRAINT FK_COLLEGE FOREIGN KEY (COLLEGE) REFERENCES COLLEGES(NAME) ON DELETE CASCADE;
ALTER TABLE APPLIES
ADD CONSTRAINT FK_SID FOREIGN KEY (SID) REFERENCES STUDENTS(ID) ON DELETE CASCADE;
-- 3. Now, complete the exercise
DELETE FROM STUDENTS WHERE ID IN(
   SELECT SID
   FROM APPLIES
   GROUP BY SID
   HAVING COUNT(DISTINCT MAJOR) > 2
   );
-- 4. What errors should have appeared if you had tried to do the exercise without the
-- first two steps? How did you have solved the problem?
```

If you were to try to execute the `DELETE` directly on the `STUDENT` table without setting `ON DELETE CASCADE`, the database engine (such as MySQL, PostgreSQL or Oracle) would stop the operation and return a **Referential Integrity Violation** error.

The message would look something like this:

> *Error: Cannot delete or update a parent row: a foreign key constraint fails.*

If you were unable to use `ON DELETE CASCADE` (which is the automatic and elegant way of doing it), you would have to follow a **manual cascade delete** strategy. The order of the steps here does make a difference:

i. **Identify the culprits:** First, you would need to find the IDs of the students who meet the condition (having applied for more than two courses).
ii. **Delete the 'child' records first:** Remove all rows in the `APPLIES` table that belong to those specific IDs.
iii. **Delete the "parent" records last:** Once the students no longer have any linked applications, the system will allow you to delete them from the `STUDENT` table without errors.

```sql
-- 44. Delete colleges with no CS applicants.
DELETE FROM COLLEGES WHERE NAME IN(
    SELECT COLLEGE
    FROM APPLIES
    WHERE MAJOR != 'CS');

-- 45. Accept applicants (set decision=true) to Stanford with mark < 3.6 but turn them into
-- majors 'EE'.
UPDATE APPLIES
SET DECISION = 1
WHERE COLLEGE = 'Stanford' AND MAJOR != 'EE'
AND SID IN(
    SELECT ID
    FROM STUDENTS
    WHERE MARK < 3.6
```

```sql
);

-- 46. Turn the highest-mark CS applicant into a bioengineering applicant.
UPDATE APPLIES
SET MAJOR = 'bioengineering'
WHERE MAJOR = 'CS'
  AND SID = (
  SELECT SID
  FROM APPLIES JOIN STUDENTS ON APPLIES.SID = STUDENTS.ID
  WHERE MAJOR = 'CS'
  ORDER BY MARK DESC
  FETCH FIRST 1 ROWS ONLY);

-- 47. Give everyone the highest mark and smallest high school.
UPDATE STUDENTS
SET MARK = (SELECT MAX(MARK) FROM STUDENTS),
    SIZE_HIGH_SCHOOL = (SELECT MIN(SIZE_HIGH_SCHOOL) FROM STUDENTS);

-- 48. Accept everyone.
UPDATE APPLIES
SET DECISION =1;

-- 49 Considering the definition of the database:
-- STUDENTS (id, name, surname, mark, size_high_school)
-- COLLEGES (name, state, enrolment)
-- APPLIES (sid*, college*, major, decision)
-- The database is not 100% normalized. Please, normalize it and explain what
-- suppositions you have made.
```

Table: `HIGH_SCHOOLS`

To comply with **2NF/3NF**, we extract the school data.

- **hs_id** (PK)
- hs_name
- size

Table: `STUDENTS`

The student table now only contains data about the individual and a reference to their school.

- **id** (PK)
- name
- surname
- mark
- *hs_id* (FK referencing `HIGH_SCHOOLS`)

Table: `COLLEGES`

This table is usually already in 3NF, unless we consider that the state has its own data.

- **name** (PK)
- state
- enrolment

Table: `MAJORS` (Optional but recommended)

To avoid typos (such as writing "CS", "CompSci" and "Computer Science"), it is best to have a glossary.

- **major_id** (PK)
- major_name

Table: `APPLIES`

The table of facts that ties everything together.

- **sid** (FK, PK part)
- **college_name** (FK, PK part)
- **major_id** (FK, PK part)
- decision